

ONNX-EdgeIDS: A Framework-Agnostic ONNX/NumPy Serving Toolkit for Streaming Intrusion Detection on Jetson Orin Nano Super

Thai Nguyen Vu¹, Dung Van Bui^{2,3}, Nhut Tri Do^{2,3}, and Van Du Nguyen⁴

¹ Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

thaivn_ph@utc.edu.vn

² Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam

³ University of Information Technology, Ho Chi Minh City, Vietnam

dungbv.17@grad.uit.edu.vn, trinhutdo@uit.edu.vn

⁴ Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

nvdu@hcmuaf.edu.vn

Abstract Earlier deployment work on this system exposed a limitation that recurs across edge intrusion detection projects: a model can be accurate while the serving stack it inherited from training remains too heavy for an 8 GB edge board. We present ONNX-EDGEIDS, a two-stage Kafka serving toolkit built around a single ONNX artifact rather than around whichever framework produced it. Its NumPy fallback exporter reads the standard `ai.onnx.ml.TreeEnsembleClassifier` and `Scaler` operators directly out of the ONNX graph, so it works for any ONNX tree-ensemble classifier, not one tied to a particular training framework; a release-gate script cross-validates the two backends and refuses to pass if label agreement or confidence drift exceeds tolerance. On a two-node Jetson Orin Nano Super cluster, the ONNX Runtime backend serves 20,185 rows/s at a p95 of 0.068 ms/row, against 152 rows/s and 8.23 ms/row for the NumPy fallback at the same batch size, a gap that narrows sharply as batch size grows (9 → 150 → 1,650 rows/s at batch 1/20/500); the NumPy artifact is nonetheless 6× smaller (1.23 vs. 7.38 MB) and reaches its first verdict 32% faster after a cold start (368 vs. 542 ms), a trade-off relevant to restart-sensitive deployments. On a validation run of 2,000 replayed flows, the two backends agree on every label with zero confidence deviation. Rather than only citing recent ARM-oriented tree-ensemble work as future work, we apply its underlying lesson – evaluate many trees in lock-step instead of one at a time – and rewrite the NumPy backend’s traversal loop accordingly: on this paper’s ARM64 development hardware, the rewrite yields up to 84× throughput at batch size 1 with no prediction-parity regression, pending re-confirmation on the Jetson boards themselves. *[We report end-to-end throughput and latency over a wired inter-board link, and the Jetson re-measurement of the traversal rewrite, the two measurements still pending at the time of writing.]*

Keywords: Intrusion detection · Edge computing · ONNX Runtime · Model serving · Jetson Orin Nano Super · Apache Kafka.

1 Introduction

Streaming ML-based intrusion detection increasingly runs at the network edge to cut bandwidth and response time [14, 5]. A recurring failure mode in such deployments is not the model but the serving stack around it: a framework convenient for offline training is reused for online serving because it is operationally simpler to run one stack than two, and that convenience carries a memory and latency cost that a constrained ARM board cannot always absorb. Earlier deployment work on this project’s two-stage anomaly-gate-then-classifier pipeline exposed exactly this limitation: the classifier was accurate, but its serving path was heavier than the 8 GB Jetson Orin Nano Super boards it ran on could comfortably sustain.

This paper presents the software response to that limitation: ONNX-EDGEIDS⁵, a serving toolkit organized around one portable artifact, an ONNX `TreeEnsembleClassifier` graph, with two interchangeable runtimes behind it. The primary runtime is ONNX Runtime; a second, dependency-minimal runtime re-implements the same forest in pure NumPy for boards where no matching `onnxruntime` wheel exists. Critically, the NumPy exporter does not depend on the framework that produced the ONNX graph: it parses the graph’s standard tree-ensemble operator attributes (split features, thresholds, per-leaf class weights) rather than a framework-specific internal representation, so the same exporter works on an ONNX tree ensemble regardless of its origin. A companion validation script treats agreement between the two runtimes as a release gate rather than a one-off sanity check, failing the build if label agreement or confidence drift falls outside tolerance.

We integrate both runtimes into a two-stage Kafka pipeline – an anomaly gate on one Jetson board forwarding suspicious flows to a classifier on a second – and report on-hardware measurements: steady-state throughput and latency at multiple batch sizes, cold-start time to first verdict, artifact size, and cross-runtime prediction parity. Several of these findings are not obvious from a single-node microbenchmark alone: the NumPy backend’s throughput is highly batch-size dependent because its tree traversal is vectorized across a batch rather than compiled per tree, and on this hardware the Kafka client library, not inference, is the binding constraint once the ONNX Runtime backend is in use.

Contributions. (1) A classifier-tier contract with two interchangeable runtimes (ONNX Runtime, pure NumPy) behind one `InferenceEngine` interface, integrated into a two-stage Kafka gate-then-classifier pipeline on a Jetson Orin Nano Super cluster; (2) A framework-agnostic NumPy exporter that reads an ONNX tree ensemble’s standard operator attributes directly, together with a release-gate parity validator that failed builds rather than only warning on divergence; (3) On-hardware measurements covering steady-state cost, batch-size sensitivity, cold-start latency, artifact size, and prediction parity, several of which surface trade-offs (cold start vs. throughput, artifact size vs. speed) that a single steady-state number would hide; (4) A lockstep tree-traversal rewrite of the NumPy backend that applies the lesson of recent ARM-oriented tree-ensemble work as a measured optimization rather than a future-work citation, up to

⁵ <https://github.com/vuthainguyen1602/onnx-edge-ids>

84× faster at batch size 1 with no prediction-parity regression (Sect. 6.3); (5) An open, archived software artifact⁶ and reproducible benchmark scripts.

2 Related Work

Edge intrusion detection. Single-board deployments of ML-based IDS are now common, from early Raspberry Pi systems [12] to recent ensemble and metaheuristic-optimized detectors on IoT-edge and industrial-control hardware [2, 11, 15] and autoencoder- or generative-model-based detection on Jetson boards [6, 3]. These studies optimize the detector; the present paper instead treats the *serving stack* as the object of study, on the observation (from earlier deployment work on this system) that model accuracy and serving cost are separate axes that a single evaluation can conflate.

Serving classical ML on constrained hardware. ONNX⁷ standardizes a small operator set, including `TreeEnsembleClassifier` for decision-tree ensembles, so a model trained in one framework can be served by any ONNX-compatible runtime; ONNX Runtime⁸ is the reference CPU/GPU implementation and is what we target as the primary backend. Nakandala et al. [10] make a related argument for tree ensembles specifically: compiling scikit-learn-style pipelines into tensor operations lets a GPU tensor runtime, rather than a tree-optimized one, serve them, with large reported speedups over native execution on batched workloads. Our NumPy fallback takes the opposite trade-off deliberately: no compilation step and no GPU dependency, at the cost of the batch-size sensitivity this trades in. Koschel et al. [7] show that this cost is not fundamental to ARM hardware: porting the QuickScorer tree-traversal algorithm from Intel AVX to ARM NEON yields up to 9.4× speedups for ensemble inference on IoT-class ARM devices, with fixed-point quantization improving throughput further at a small accuracy cost. We do not port QuickScorer’s bitvector algorithm or write NEON intrinsics; we do take its underlying lesson – evaluate many trees in lockstep rather than one at a time – and apply it at the NumPy level to `NumpyInferenceEngine`’s traversal loop, with a measured result rather than a citation (Sect. 6.3).

ONNX and quantization in network intrusion detection. Model compilation and quantization for ONNX-served detectors is an active line of work in this application domain specifically: Eren et al. [4] report a systematic comparison of ONNX compilation and quantization configurations for DrDoS/amplification-attack detectors (MLP, CNN, GRU architectures), showing that quantized ONNX export can materially change throughput and latency for network-security classifiers without simply assuming DNN-quantization results transfer unchanged. Their target model family is different from the tree ensemble served here, but the result establishes that ONNX-level quantization is a validated, actively studied lever in exactly this application space, and motivates evaluating it against our tree-ensemble graph rather than the deep architectures it was demonstrated on.

Evaluation protocol. Arp et al. [1] and Lanvin et al. [8] document how easily an offline security-ML evaluation is inflated by leakage or dataset artifacts; the

⁶ Zenodo concept DOI: <https://doi.org/10.5281/zenodo.22731927>.

⁷ <https://onnx.ai/>

⁸ <https://onnxruntime.ai/>

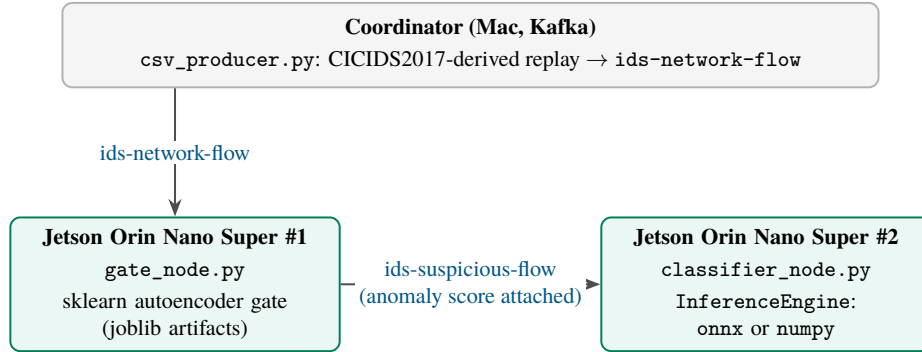


Fig. 1. Two-stage Kafka pipeline: gate on Jetson #1, swappable ONNX/NumPy classifier on Jetson #2. This minimal toolkit prints per-batch metrics and an optional CSV; the PostgreSQL/alerting/Grafana stack of the earlier deployment work is out of scope here.

classifier served by this toolkit was selected under a leakage-aware, deduplicated CICIDS2017 [13] protocol with SHAP-based [9] feature selection in earlier work on this system, out of scope for the present paper. What is in scope here is a different correctness question: whether the *exported serving artifacts* reproduce the classifier they were exported from. Sect. 4 treats that as a release-gated, measured property rather than an assumption.

3 System Architecture

3.1 Two-Stage Kafka Pipeline

Fig. 1 shows the deployment topology. A coordinator host runs Kafka and replays CICIDS2017-derived flow records onto an input topic at a configurable rate. Jetson #1 runs the anomaly gate: a scikit-learn autoencoder and scaler (loaded via joblib, trained offline and out of scope here) score each batch, and flows whose reconstruction error clears a calibrated threshold are forwarded to a suspicious-flow topic with their anomaly score attached. Jetson #2 consumes that topic and classifies each flow with the swappable `InferenceEngine` described below. Both gate and classifier batch their Kafka consumption with the same policy: a batch is served either once it reaches a target size or once a linger timeout elapses since its first record, whichever comes first, so a thin stream still gets bounded latency instead of waiting indefinitely to fill a batch.

3.2 Swappable Classifier Tier

- `FeatureMatrixBuilder` turns a batch of Kafka flow dicts into a dense float32 matrix in a fixed column order, mapping missing/NaN/Inf values to zero.
- `InferenceEngine` is the shared contract: `create_inference_engine(engine, model_path, providers)` returns either backend, and both return a common `BatchPrediction` (predictions, per-row confidences, batch timing/attack-count stats), so `classifier_node.py` never branches on which backend is active.

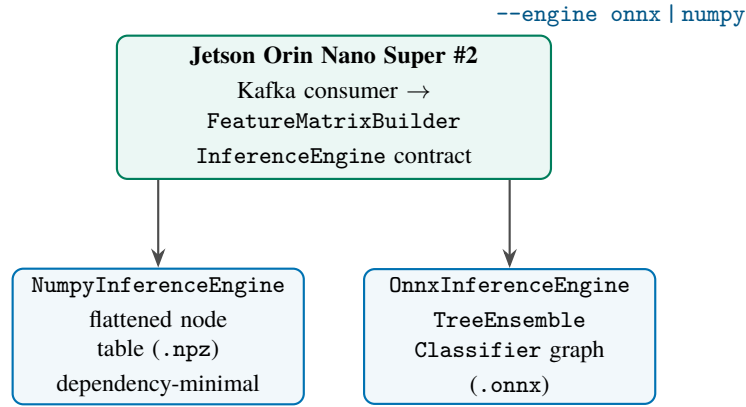


Fig. 2. Swappable classifier tier on Jetson #2: one feature-matrix builder, one `InferenceEngine` contract, two backends selected by a single flag.

- `OnnxInferenceEngine` loads the `.onnx` artifact with ONNX Runtime’s CPU execution provider (GPU/TensorRT providers are selectable but not evaluated in this paper).
- `NumpyInferenceEngine` loads the flattened forest described in Sect. 4 and walks every tree with a vectorized loop over an active-row boolean mask: at each step, active rows look up their current node’s split feature and threshold, move left or right, and rows that land on a leaf accumulate that leaf’s class scores and drop out of the mask.

Fig. 2 shows the contract. Selecting a backend is a single `--engine onnx|numpy` flag to `classifier_node.py`; nothing else in the pipeline changes.

4 Model Export and Conversion Correctness

4.1 Framework-Agnostic NumPy Export

The NumPy exporter (`scripts/export_numpy.py`) treats the ONNX file as the single source of truth: it loads the graph with the `onnx` package and reads the `ai.onnx.ml.Scaler` node’s `offset/scale` attributes and the `ai.onnx.ml.TreeEnsembleClassifier` node’s `nodes_treeids`, `nodes_nodeids`, `nodes_featureids`, `nodes_values`, `nodes_modes`, `nodes_truenodeids`, `nodes_falsenodeids`, `classlabels_int64s`, `class_treeids`, `class_nodeids`, `class_ids`, and `class_weights` attributes – all part of the ONNX operator specification, not of any particular training framework’s internal format. It currently supports the `BRANCH_LEQ` split mode only and raises rather than silently mis-converting on any other mode, so a future export from a framework using a different split convention fails loudly instead of producing a plausible-looking but wrong artifact. Because the exporter never touches how the ONNX graph was produced, the same code exports a NumPy fallback for any ONNX tree ensemble that uses this operator, independent of its training framework.

A second correctness check is feature-order agreement: if the source ONNX model’s `doc_string` carries a JSON `feature_columns` list (as this project’s exporter embeds at ONNX-export time), `export_numpy.py` verifies it matches the externally supplied `feature_columns.json` and refuses to export on a mismatch, since a silently permuted feature order would produce confident, wrong predictions rather than an obvious failure.

4.2 Parity as a Release Gate

`scripts/validate_parity.py` runs both backends over the same batch-built feature matrix and reports label agreement and the maximum confidence deviation between them; it exits non-zero if agreement falls below a configurable threshold (default: exact, 1.0) or the deviation exceeds a tolerance (default 10^{-5}), so it is usable as a CI-style gate rather than only a manual check, and it optionally writes a machine-readable JSON report. On a validation run of 2,000 replayed CICIDS2017-derived flows, the two backends agreed on every label (2,000/2,000) with a maximum confidence deviation of 0.0, i.e. bit-identical decisions on the sampled flows; Table 1 reports this alongside artifact size.

Table 1. Export validation: NumPy-vs-ONNX prediction parity on a replay sample.

Metric	Value
Replay rows validated	2,000
Label agreement	2,000 / 2,000 (100.0000%)
Max confidence deviation	0.0
ONNX artifact size	7.38 MB
NumPy artifact size	1.23 MB (6.0× smaller)

Source: `scripts/validate_parity.py` on a single replay run (this software’s own submission-checklist validation run); re-run on a held-out sample as a submission gate rather than treated as a one-time result.

5 Experimental Setup

The testbed is two NVIDIA Jetson Orin Nano Super Developer Kits (6-core Arm Cortex-A78AE CPU, Ampere GPU, 8 GB LPDDR5) and a coordinator host running Kafka, over a LAN; at the time of writing, Jetson #2’s inter-board link is Wi-Fi rather than wired, which affects only the end-to-end transport figures still marked pending below, not the in-process measurements in Sect. 6. `scripts/csv_producer.py` replays CICIDS2017-derived [13] flow rows onto the input topic at a configurable rate; `gate_node.py` runs on Jetson #1 and `classifier_node.py` on Jetson #2, the latter re-run once per backend (`--engine onnx` and `--engine numpy`) with the gate held fixed, and once more across a batch-size sweep (1, 20, 500) to characterize batch

sensitivity. Reported metrics: throughput (rows/s), p95 per-row latency (ms), resident memory (RSS), board power (W, `tegrastats`, not idle-subtracted, so figures are total-board rather than active-only), cold-start time to first verdict, and the parity metrics of Sect. 4.2.

6 Results

6.1 Steady-State Cost per Backend

Table 2 reports classifier-node cost at batch size 20 for each backend, with the gate’s own cost reported separately since it runs a different model (an autoencoder, not the forest). ONNX Runtime sustains roughly $133\times$ the NumPy backend’s throughput at this batch size and a fraction of its p95 latency, at the cost of higher resident memory and marginally lower measured board power (4.64 vs. 4.93 W; since power is not idle-subtracted, this difference should be read cautiously rather than as a per-inference energy claim). The NumPy row here is the original per-tree traversal, measured on the Jetson boards; Sect. 6.3 rewrites that traversal and measures the rewrite on different (ARM64 development) hardware, so the two are not yet merged into one table.

Table 2. Classifier-node steady-state cost per backend (batch size 20) and gate cost, single validation run on Jetson hardware.

Node / engine	Throughput (rows/s)	p95 (ms/row)	RSS (MB)	Power (W)
Classifier – ONNX Runtime	20,185	0.0676	155.6	4.64
Classifier – NumPy	152	8.2267	46.2	4.93
Gate – sklearn autoencoder	13,915	0.0783	[TBD]	[TBD]

Power is total-board `tegrastats` power, not idle-subtracted. Gate throughput/latency reported at its own default batch size (100); its forward rate to the classifier topic was 28.64% on this replay sample. Source: `submission_checklist.md` in the software repository, single validation run.

6.2 NumPy Batch-Size Sensitivity

The original `NumpyInferenceEngine` traverses each tree with its own Python-level while-loop, one tree at a time; because each such loop’s per-call NumPy dispatch overhead is paid once per tree regardless of how little work that call does, its throughput scales strongly with batch size on Jetson hardware: 9 rows/s at batch 1, 150 rows/s at batch 20, and 1,650 rows/s at batch 500 on the same board (`submission_checklist.md`, this software’s own validation run). ONNX Runtime is comparatively insensitive to batch size over the same range (Sect. 7). This means the two backends should not be compared at a single fixed batch size without stating it, and a deployment choosing the NumPy fallback should size its batch/linger policy accordingly rather than default to a small batch tuned for the ONNX path – or, as Sect. 6.3 shows, the per-tree loop itself is the thing to fix.

6.3 A Lockstep Traversal Optimization

The batch-size sensitivity in Sect. 6.2 is a direct, fixable symptom of the per-tree Python loop, not an inherent property of a NumPy-only backend. Node ids are already globally unique across the flattened forest (`tree_offsets` partitions the flat node arrays), so nothing requires visiting trees one after another: we rewrote `NumpyInferenceEngine._infer` to hold a $(n_{\text{trees}}, n_{\text{rows}})$ matrix of current node ids and advance *every* tree for *every* row in each round, stopping a (tree, row) pair only once it reaches a leaf. This turns the Python-level loop from $O(n_{\text{trees}} \times \text{depth})$ iterations into $O(\text{depth})$ iterations – on the 200-tree forest served here, the lockstep loop converges in 16 rounds regardless of tree count, against up to 200 independent per-tree loops before.

We validated the change two ways. First, correctness: on 5,000 replayed rows the rewritten engine matches the original engine’s predictions exactly (5,000/5,000 labels) with a maximum raw-score deviation of 3.6×10^{-7} , attributable to floating-point summation order rather than a logic change; re-running the software’s own release-gate script, `validate_parity.py`, against the ONNX artifact after the rewrite still passes at its default tolerance (2,000/2,000 label agreement, 3.6×10^{-7} max confidence delta, against a 10^{-5} gate). Second, speed: Table 3 reports throughput before and after, measured on this paper’s development machine (Apple M3, ARM64) at the same three batch sizes as Sect. 6.2, best of three runs.

Table 3. Per-tree loop vs. lockstep traversal, same artifact, same machine (Apple M3, ARM64; not yet re-measured on the Jetson Cortex-A78AE boards – see limitations below).

Batch size	Per-tree loop (rows/s)	Lockstep (rows/s)	Speedup
1	45.2	3,805.9	84.2×
20	492.7	12,124.1	24.6×
500	7,162.2	13,125.2	1.8×

The gain is largest exactly where the original backend was worst – small batches, where per-call dispatch overhead previously dominated actual computation – and shrinks as batch size grows and the per-tree loop’s overhead is already amortized over more rows. This is not a port of QuickScorer or NEON intrinsics (Sect. 3 related work); it is a plain algorithmic restructuring that removes the dispatch overhead QuickScorer-style approaches also target, measured rather than assumed. Two limitations qualify it: the numbers in Table 3 come from an Apple M3 laptop, not yet the Jetson Cortex-A78AE boards the rest of this paper measures on, so the algorithmic argument (fewer Python-level loop iterations) should transfer but the absolute speedup factors may not; and `np.add.at`, used to scatter-accumulate leaf scores from multiple trees into the same row’s score vector, is not the fastest NumPy accumulation primitive available, so a further pass replacing it with a sorted-reduction could plausibly add more.

6.4 Cold Start: Time to First Verdict

Table 4 breaks down the time from process start to the first served verdict. ONNX Runtime pays a larger one-time session-construction cost (294.5 ms) than NumPy’s artifact load (79.7 ms), but performs its own first prediction fastest (3.1 ms), consistent with the compiled-runtime-vs-interpreted-loop trade-off in Table 2; on this run NumPy still reaches its first verdict sooner overall (368.4 vs. 542.3 ms), since its faster load dominates over the whole warm-up path. This favors NumPy specifically for restart-sensitive scenarios (e.g. frequent container restarts or fast failover) even though ONNX Runtime is far ahead on steady-state throughput.

Table 4. Cold-start latency breakdown per backend (mean of both boards, page cache dropped).

Engine	Load/session build (ms)	First prediction (ms)	Time to first verdict (ms)
ONNX Runtime	294.5	3.1	542.3
NumPy	79.7	115.1	368.4

7 Discussion

The Kafka client, not inference, is now the binding constraint. Once the ONNX Runtime backend is in use, classifier-node throughput (20,185 rows/s) is an order of magnitude above what the pure-Python `kafka-python` client sustains on the same board (roughly 2,500 msg/s consuming, 2,200 msg/s producing; the identical code reaches 6,600/9,900 msg/s on a laptop). Closing this gap is a client-library problem, not an inference problem: a C-backed client such as `confluent-kafka` is the next lever, and reporting an end-to-end (producer-timestamp-to-verdict) throughput number without controlling for this would misattribute a Kafka-client ceiling to the classifier.

ONNX vs. NumPy is a genuine trade-off, not a strict ordering. Table 2 makes ONNX Runtime look like the only reasonable choice, but Table 4 and the artifact-size row of Table 1 complicate that: the NumPy artifact is $6\times$ smaller and reaches its first verdict sooner, which matters on boards where `onnxruntime` cannot be installed at all, or where restart frequency dominates steady-state throughput. The toolkit’s role is to make that trade-off a one-flag choice rather than a rewrite.

Applying recent tree-ensemble-on-ARM work to the NumPy backend. Sect. 6.3 applies the underlying lesson of Koschel et al.’s ARM NEON port of QuickScorer [7] – evaluate many trees together rather than one at a time – as a lockstep NumPy traversal, and measures rather than assumes the result: up to $84\times$ at batch size 1 on this paper’s development hardware, with no prediction-parity regression. What we did not do is port QuickScorer’s bitvector algorithm itself or write NEON intrinsics; Koschel et al.’s reported $9.4\times$ figure is for that specific SIMD-level technique on different ARM IoT hardware; a NEON-intrinsics implementation, and quantization on top of it as they also report, remains a further, larger step than the one taken here. Re-measuring Table 3 on

the Jetson Cortex-A78AE boards used elsewhere in this paper, rather than the development machine it was measured on, is the immediate next step.

Quantizing the ONNX graph itself. Eren et al. [4] demonstrate that ONNX-level quantization measurably changes throughput and latency for network-security classifiers, on DNN architectures rather than tree ensembles. Whether the same holds for a `TreeEnsembleClassifier` graph (whose costs are dominated by branching and memory access, not the matrix multiplies quantization typically targets) is an open, testable question for this toolkit rather than an assumption we import; we list it as future work.

Limitations. All steady-state, cold-start, and power figures in Sect. 6 come from a single validation run under one set of board conditions; `submission_checklist.md`’s own acceptance criteria (Sect. 4) call for repeated runs, which this draft has not yet incorporated. Power is total-board `tegrastats` power without an idle baseline subtracted, so it should not be read as per-inference energy. End-to-end throughput and latency over a wired inter-board link – as opposed to the current Wi-Fi link – are not yet measured; *[fill in once available, following the protocol in papers/onnx2026/README.md]*. The lockstep traversal of Sect. 6.3 is measured on this paper’s ARM64 development machine, not yet on the Jetson boards Table 2 and Table 4 report on; *[re-measure Table 3 on the Jetson Cortex-A78AE boards and fold the result into Table 2 once available]*. This toolkit’s classifier node does not yet write verdicts to a database or raise alerts; that integration exists in the earlier deployment work’s stack and is out of scope for the software paper this toolkit accompanies, but end-to-end latency including a persistence step is not covered here either.

8 Conclusion

We presented ONNX-EDGEIDS, a two-runtime serving toolkit for a two-stage Kafka edge IDS pipeline that treats an ONNX tree-ensemble graph, not the framework that produced it, as the portable artifact. Its NumPy fallback exporter reads standard ONNX tree-ensemble operator attributes directly, so the same exporter generalizes across training frameworks, and a release-gate script measures rather than assumes agreement between the two runtimes: 2,000/2,000 label agreement with zero confidence deviation on the validation sample reported here. On-hardware measurements show ONNX Runtime $133\times$ faster than the original NumPy fallback at batch size 20, a gap that narrows sharply at larger batch sizes, while the NumPy artifact is $6\times$ smaller and reaches its first verdict 32% faster from a cold start – a genuine trade-off rather than a strict ordering. We then closed part of that gap rather than only noting it: rewriting the NumPy backend’s per-tree Python loop into a lockstep traversal over all trees at once yields up to $84\times$ at batch size 1 with no prediction-parity regression (Sect. 6.3), applying the lesson of recent ARM-oriented tree-ensemble work [7] as a measured result rather than a citation, though re-confirmed on the development machine rather than the Jetson boards themselves so far. Once the ONNX backend is in use, the pure-Python Kafka client, not inference, becomes the binding throughput constraint on this hardware. *[The remaining work before this paper is submission-ready is empirical: measure end-to-end throughput over a wired inter-board link, re-measure the lockstep traversal on the Jetson Cortex-A78AE boards, and repeat the steady-state/cold-start measurements across*

multiple runs rather than the single validation run reported here.] Future work beyond that includes NEON intrinsics and fixed-point quantization for the lockstep traversal as in [7], evaluating ONNX-level quantization on the tree-ensemble graph [4], and a confluent-kafka-based client to remove the Kafka-client ceiling identified in Sect. 7.

Disclosure of Interests. The authors declare that they have no competing interests.

References

1. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 3971–3988 (2022)
2. Chithra Rani, P.R., Baalaji, K.: Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms. *Discover Applied Sciences* **7**, 928 (2025). <https://doi.org/10.1007/s42452-025-06871-z>
3. Diaz-Gorin, J., Caballero-Gil, C., Brankovic, L.: Enhancing network security with generative AI on Jetson Orin Nano. *Applied Sciences* **16**(3), 1442 (2026). <https://doi.org/10.3390/app16031442>
4. Eren, S., Gültekin, A., Özkan, Ö., Özçelik, İ.: High-throughput and low-latency DrDoS detection using quantized ONNX models. *Symmetry* **18**(7), 1187 (2026). <https://doi.org/10.3390/sym18071187>
5. Hamdan, S., Ayyash, M., Almajali, S.: Edge-computing architectures for internet of things applications: A survey. *Sensors* **20**(22), 6441 (2020). <https://doi.org/10.3390/s20226441>
6. Hasan, T., Hossain, A., Ansari, M.Q., Syed, T.H.: Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning (2025), arXiv:2501.15266
7. Koschel, S., Buschjäger, S., Lucchese, C., Morik, K.: Fast inference of tree ensembles on ARM devices (2023), arXiv:2305.08579
8. Lanvin, M., Gimenez, P.F., Han, Y., Majorczyk, F., Mé, L., Totel, É.: Errors in the CI-CIDS2017 dataset and the significant differences in detection performances it makes. In: *Risks and Security of Internet and Systems (CRiSIS 2022)*, Revised Selected Papers. pp. 18–33. *Lecture Notes in Computer Science*, Springer (2023). https://doi.org/10.1007/978-3-031-31108-6_2
9. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems* **30** (2017)
10. Nakandala, S., Saur, K., Yu, G.I., Karanasos, K., Curino, C., Weimer, M., Interlandi, M.: A tensor compiler for unified machine learning prediction serving. In: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association (2020)
11. Nasir, K., Badri, S.K., Alghazzawi, D.M., Alghamdi, M.Y., Alkhozae, M., Almakky, A., Alhazmi, R.M., Asghar, M.Z.: HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning. *Scientific Reports* **16**, 2313 (2026). <https://doi.org/10.1038/s41598-025-32183-8>
12. Sforzin, A., Gómez Mármol, F., Conti, M., Bohli, J.M.: RPiDS: Raspberry pi IDS — a fruitful intrusion detection system for IoT. In: *Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom)*. pp. 440–448 (2016). <https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCoM-IoP-SmartWorld.2016.0080>
13. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. pp. 108–116 (2018)

14. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. *IEEE Internet of Things Journal* **3**(5), 637–646 (2016). <https://doi.org/10.1109/IIOT.2016.2579198>
15. Zhang, Z., Zhang, L., Xu, J., Chen, Z., Wang, P.: An edge-deployable lightweight intrusion detection system for industrial control. *Electronics* **15**(3), 644 (2026). <https://doi.org/10.3390/electronics15030644>